

BCaaS

Résumé

Ce rapport présente la conception et le développement de BCAAS (*Business Core as a Service*), une plateforme générique multi-tenant dédiée à la gestion standardisée des processus métier. Le projet s'inscrit dans le cadre du cours d'Introduction aux Réseaux de la troisième année de Génie Informatique.

L'idée centrale est de modéliser un système d'information métier en s'appuyant rigoureusement sur les concepts de l'ingénierie des protocoles réseaux : pile de couches, encapsulation, routage, adressage, qualité de service et plans de contrôle et de données. Cette analogie formelle permet de concevoir une architecture microservice cohérente, modulaire et extensible.

La solution implémentée repose sur Spring Boot 3.5 / Java 21, PostgreSQL avec Liquibase, Spring Security avec JWT, et SpringDoc OpenAPI pour la documentation automatique de l'API. Le frontend est développé avec Next.js sous forme de Progressive Web App (PWA).

Mots-clés : Business Core, multi-tenant, microservices, protocoles réseaux, OSI, encapsulation, Spring Boot, API REST, Next.js.

Table des matières

Résumé	1
1 Introduction générale	6
1.1 Contexte du projet	6
1.2 Problématique	6
1.3 Objectifs	7
1.4 Structure du rapport	7
2 État de l’art	8
2.1 Plateformes métier génériques	8
2.2 Architectures multi-tenant	8
2.3 Ingénierie des protocoles réseaux comme méthode	9
3 Analogie formelle entre réseaux et métier	10
3.1 Formalisation mathématique	10
3.2 Table de correspondance réseau-métier	11
3.3 Modélisation de la requête comme datagramme	11
3.4 Flux de traitement comme pipeline de protocoles	11
3.5 Modélisation du multi-tenant comme réseau virtuel	12
3.6 Control plane et data plane	12
4 Architecture globale de la solution	14
4.1 Vue d’ensemble	14
4.2 Description des couches	14
4.2.1 Couche 1 — Infrastructure	14
4.2.2 Couche 2 — Transport & Messaging	15
4.2.3 Couche 3 — Tenant & Routing	15
4.2.4 Couche 4 — Context & Policy	15
4.2.5 Couche 5 — Business Capabilities	15
4.3 Composants de l’architecture microservice	15

5	Conception détaillée	17
5.1	Modèle de données générique	17
5.1.1	Entités principales	17
5.1.2	Diagramme entité-relation simplifié	17
5.2	Conception de l'API REST	17
5.3	Sécurité et authentification	18
6	Implémentation technique	20
6.1	Pile technologique	20
6.2	Structure du projet Spring Boot	20
6.3	Filtre de résolution du tenant	21
6.4	Structure du paquet métier (DTO)	22
6.5	Pertinence de Kafka et Redis dans BCAAS	23
7	Frontend Next.js	24
7.1	Architecture du frontend	24
7.2	Choix techniques	24
7.3	Pages principales	25
7.3.1	Site vitrine	25
7.3.2	Tableau de bord admin	25
8	Tests et validation	26
8.1	Stratégie de tests	26
8.2	Scénario de test de provisionnement tenant	26
9	Conclusion et perspectives	28
9.1	Bilan	28
9.2	Perspectives	28
9.3	Apports personnels	29

Table des figures

3.1	Analogie structurelle entre un datagramme IP et un paquet métier BCAAS	11
3.2	Flux bout-en-bout d'une requête dans BCAAS — analogie OSI	11
4.1	Pile protocolaire métier de BCAAS et correspondance avec le modèle OSI	14
4.2	Architecture microservice de référence de BCAAS	16
5.1	Diagramme entité-relation du modèle générique	18

Liste des tableaux

2.1	Stratégies de multi-tenancy	9
3.1	Correspondance formelle entre concepts réseaux et concepts métier . . .	13
5.1	Entités du modèle de données générique	17
5.2	Endpoints principaux de l'API BCAAS	18
6.1	Pile technologique de BCAAS	20
7.1	Technologies frontend	24

Chapitre 1

Introduction générale

1.1 Contexte du projet

Dans le domaine du génie logiciel, chaque nouveau projet métier — qu’il s’agisse d’une pharmacie, d’une agence immobilière, d’un cabinet médical ou d’une école — reconstruit invariablement les mêmes fondations : gestion des acteurs, des ressources, des règles, des workflows, des notifications et de la traçabilité. Cette duplication systématique représente un gaspillage considérable en temps de développement, en maintenance et en cohérence architecturale.

Parallèlement, l’ingénierie des réseaux a résolu depuis des décennies un problème structurellement similaire : faire communiquer de façon fiable, sécurisée et interopérable des entités hétérogènes sur une infrastructure partagée. Le modèle OSI et la suite TCP/IP proposent une décomposition en couches aux responsabilités clairement délimitées, un mécanisme d’encapsulation des données, un adressage précis des destinataires, une qualité de service différenciée et une séparation nette entre plan de contrôle et plan de données.

1.2 Problématique

Question directrice

Comment concevoir un noyau métier générique, réutilisable et configurable qui permette à n’importe quelle instance métier de fonctionner sans reconstruire les fondations, en s’appuyant formellement sur les concepts de l’ingénierie des protocoles réseaux ?

1.3 Objectifs

Les objectifs du projet sont les suivants :

1. Établir une analogie formelle et rigoureuse entre les concepts réseaux (couches OSI, encapsulation, routage, QoS, multi-tenant) et les composants d'une architecture métier générique.
2. Concevoir une architecture microservice multi-tenant en cinq couches protocolaires.
3. Implémenter un API de provisionnement de tenants avec modèle de données générique configurable.
4. Documenter et exposer l'API via Swagger UI versionné.
5. Développer un frontend Next.js comprenant un site vitrine et un tableau de bord d'administration.

1.4 Structure du rapport

Ce rapport est organisé en sept chapitres. Le chapitre 2 dresse l'état de l'art des plateformes génériques et des architectures multi-tenant. Le chapitre 3 formalise mathématiquement l'analogie entre réseaux et métier. Le chapitre 4 présente l'architecture globale de la solution. Le chapitre 5 détaille la conception des composants. Le chapitre 6 décrit l'implémentation technique. Le chapitre 7 présente le frontend. Enfin, le chapitre 9 conclut et ouvre sur des perspectives d'évolution.

Chapitre 2

État de l'art

2.1 Plateformes métier génériques

L'approche consistant à factoriser la logique commune de plusieurs domaines métier dans une plateforme partagée n'est pas nouvelle. On distingue plusieurs catégories :

ERP (Enterprise Resource Planning) Systèmes monolithiques couvrant plusieurs modules métier (SAP, Oracle EBS). Ils souffrent d'une rigidité importante et d'un couplage fort entre domaines.

Low-code / No-code platforms Plateformes comme OutSystems ou Mendix permettant de générer des applications à partir de configurations. Elles limitent cependant l'expressivité et le contrôle architectural.

Backend-as-a-Service (BaaS) Services cloud (Firebase, Supabase, Appwrite) offrant des fonctionnalités communes (authentification, base de données, stockage). Ils ne modélisent pas la sémantique métier.

BCaaS Notre approche se positionne entre le BaaS technique et l'ERP fonctionnel : un *Business Core* générique, configurable par domaine, exposé comme un service API.

2.2 Architectures multi-tenant

Le multi-tenancy désigne la capacité d'un système à servir plusieurs clients (tenants) de façon isolée sur une infrastructure partagée. Trois stratégies principales existent :

BCAAS adopte une stratégie hybride : colonne `tenant_id` pour les entités partagées et schéma séparé optionnel pour les tenants à haute exigence d'isolation.

TABLE 2.1 – Stratégies de multi-tenancy

Stratégie	Isolation	Coût	Complexité
Base de données séparée	Très forte	Élevé	Faible
Schéma séparé (PostgreSQL)	Forte	Moyen	Moyenne
Colonne <code>tenant_id</code>	Logique	Faible	Élevée

2.3 Ingénierie des protocoles réseaux comme méthode

Le modèle OSI (Open Systems Interconnection) définit sept couches, chacune offrant des services à la couche supérieure et consommant ceux de la couche inférieure. Cette décomposition présente quatre propriétés fondamentales exploitées dans notre conception :

1. **Séparation des responsabilités** : chaque couche a une fonction clairement délimitée.
2. **Encapsulation** : les données traversent les couches en s'enrichissant de méta-données sans que le contenu applicatif soit altéré.
3. **Interopérabilité** : les interfaces entre couches sont standardisées (protocoles).
4. **Substitution** : une couche peut être remplacée sans impacter les autres.

Chapitre 3

Analogie formelle entre réseaux et métier

3.1 Formalisation mathématique

Nous proposons une formalisation de l'analogie réseau-métier à travers une structure algébrique.

Définition 3.1 (Pile protocolaire métier). Soit $\mathcal{P} = (L_1, L_2, L_3, L_4, L_5)$ une pile protocolaire métier ordonnée telle que L_i offre des services à L_{i+1} et consomme les services de L_{i-1} , avec :

$L_1 =$ Infrastructure	(analogie : couche physique / liaison)
$L_2 =$ Transport & Messaging	(analogie : couche transport)
$L_3 =$ Tenant & Routing	(analogie : couche réseau)
$L_4 =$ Context & Policy	(analogie : couche session / présentation)
$L_5 =$ Business Capabilities	(analogie : couche application)

Définition 3.2 (Paquet métier). Un paquet métier $\mathcal{M}$ est un triplet :

$$\mathcal{M} = (H_T, H_C, \mathcal{P}_{biz})$$

où :

- $H_T = \{h_i \mid h_i \in \mathcal{A}_T\}$ est l'en-tête de transport contenant les attributs d'acheminement,
- $H_C = \{(k, v) \mid k \in \mathcal{K}_C, v \in \mathcal{V}\}$ est l'en-tête de contexte métier (tenant, acteur, traçabilité, SLA),

— $\mathcal{P}_{biz}$ est le payload métier : données propres à l'opération.

Définition 3.3 (Tenant). Un tenant T_i est un sextuplet :

$$T_i = (\text{id}_i, \mathcal{D}_i, \mathcal{R}_i, \mathcal{A}_i, \mathcal{W}_i, \mathcal{Q}_i)$$

où $\mathcal{D}_i$ est le domaine métier, $\mathcal{R}_i$ l'ensemble des règles, $\mathcal{A}_i$ le catalogue d'acteurs, $\mathcal{W}_i$ les workflows et $\mathcal{Q}_i$ le niveau de QoS contractualisé.

3.2 Table de correspondance réseau-métier

3.3 Modélisation de la requête comme datagramme

Considérons une requête d'ouverture de dossier. Dans un réseau classique, un datagramme IP contient des champs d'en-tête (adresse source, destination, TTL, protocole) suivis d'un payload. Dans BCAAS, la requête HTTP entrante suit la même structure :

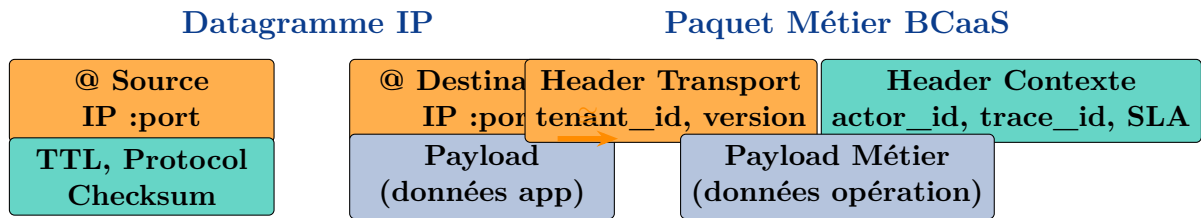


FIGURE 3.1 – Analogie structurelle entre un datagramme IP et un paquet métier BCaaS

3.4 Flux de traitement comme pipeline de protocoles

Le traitement d'une requête dans BCAAS suit exactement le modèle OSI descendant (émetteur) puis ascendant (récepteur) :

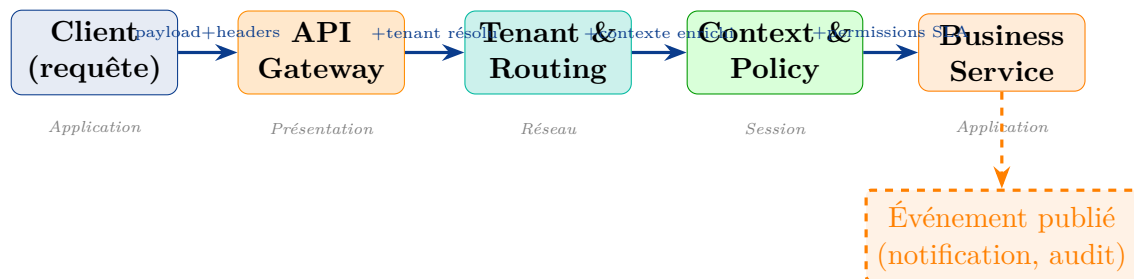


FIGURE 3.2 – Flux bout-en-bout d'une requête dans BCAAS — analogie OSI

3.5 Modélisation du multi-tenant comme réseau virtuel

Analogie : VLAN ↔ Tenant

Dans un réseau, un VLAN isole logiquement plusieurs groupes de machines sur la même infrastructure physique. Chaque trame VLAN porte un tag (802.1Q) qui identifie son appartenance. Dans BCAAS, chaque requête porte un `tenant_id` qui isole les données, les règles et les workflows d'un client sur l'infrastructure partagée.

VLAN tag $\equiv$ `tenant_id`, Switch L2 $\equiv$ Tenant Resolver, Réseau physique $\equiv$ Infrastructure pa

La propriété d'isolation peut être formalisée ainsi. Soient T_i et T_j deux tenants distincts ($i \neq j$). On exige :

$$\forall r \in \mathcal{R}, \text{tenant}(r) = T_i \implies r \notin \mathcal{V}_{T_j}$$

où $\mathcal{V}_{T_j}$ est l'ensemble des ressources visibles par le tenant T_j . Cette propriété est garantie par la présence systématique de la contrainte `tenant_id = :tid` dans toutes les requêtes SQL.

3.6 Control plane et data plane

Analogie : SDN ↔ BCaaS Admin

Dans les réseaux définis par logiciel (SDN), le control plane configure les règles de routage de façon centralisée, tandis que le data plane achemine les paquets. Cette séparation permet de modifier le comportement du réseau sans interruption de service.

Dans BCAAS, l'interface d'administration (control plane) configure tenants, règles métier, SLA et workflows. Le moteur d'exécution (data plane) traite les transactions en temps réel en appliquant ces configurations.



TABLE 3.1 – Correspondance formelle entre concepts réseaux et concepts métier

Concept réseau	Définition réseau	Équivalent métier dans BCaaS
Adresse IP	Identifiant unique d'un nœud	<code>tenant_id</code> + <code>domain</code> + <code>service</code>
En-tête de paquet	Métadonnées d'acheminement	<code>tenant_id</code> , <code>actor_id</code> , <code>trace_id</code> , <code>SLA</code>
Payload / SDU	Données applicatives	Corps de la requête métier
Encapsulation	Ajout d'en-têtes à chaque couche	Enrichissement du contexte au passage des couches
Routage	Sélection du chemin vers la destination	Résolution <code>tenant</code> → <code>service</code> → <code>instance</code>
NAT / Proxy	Traduction d'adresses	API Gateway : réécriture et normalisation
VLAN	Isolation logique sur réseau partagé	Namespace / schéma par <code>tenant</code>
TCP (fiable)	Transport avec accusé de réception	REST synchrone avec <code>retry</code> et <code>idempotence</code>
UDP (rapide)	Transport sans garantie	Événements asynchrones (Kafka)
Session TCP	État d'une connexion	Contexte de <code>saga</code> , corrélation de processus
QoS	Priorité et garanties de bande passante	SLA par <code>tenant</code> , priorité de transaction
TTL (Time To Live)	Durée de vie d'un paquet	Expiration de token <code>JWT</code> , <code>timeout</code> de <code>saga</code>
Control plane	Gestion des routes (BGP, OSPF)	Admin : configuration <code>tenants</code> , règles, SLA
Data plane	Acheminement des paquets	Exécution des transactions métier
Firewall	Filtrage de paquets	Spring Security, contrôle d'accès API
DNS	Résolution nom → adresse	Service Discovery (résolution <code>tenant</code> → <code>config</code>)

Chapitre 4

Architecture globale de la solution

4.1 Vue d'ensemble

L'architecture de BCAAS est organisée autour de cinq couches protocolaires, directement inspirées du modèle OSI, et d'une séparation claire entre control plane et data plane.

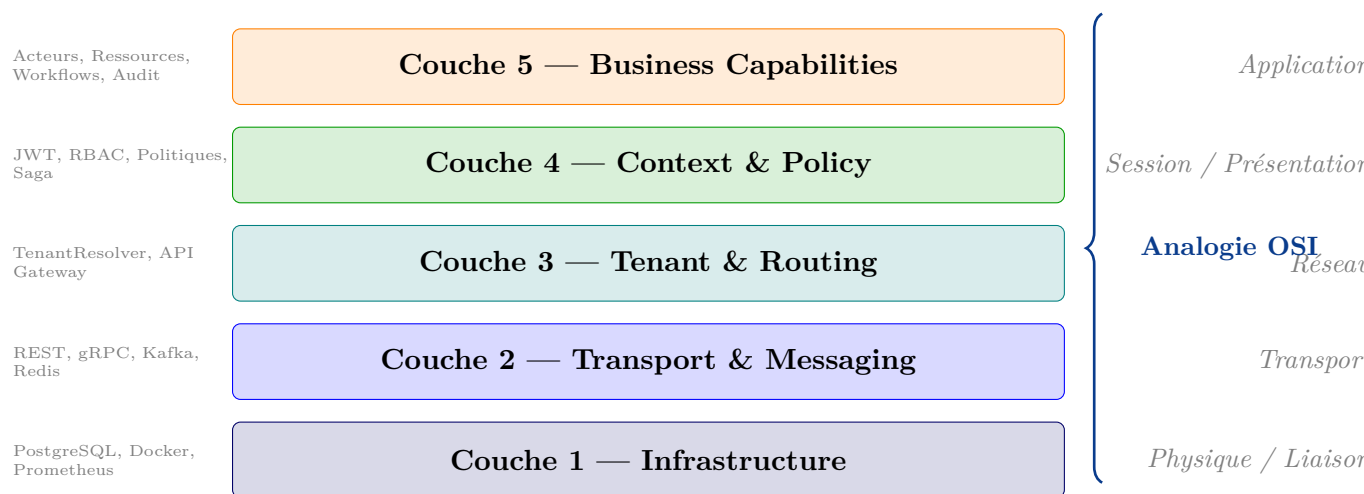


FIGURE 4.1 – Pile protocolaire métier de BCAAS et correspondance avec le modèle OSI

4.2 Description des couches

4.2.1 Couche 1 — Infrastructure

Cette couche constitue le substrat technique. Elle regroupe la base de données PostgreSQL 16 avec ses migrations Liquibase, le cache Redis, le bus de messages Apache Kafka, le stockage objet MinIO et l'ensemble des outils d'observabilité (Prometheus,

Grafana). Elle correspond aux couches physique et liaison du modèle OSI : elle transporte les données sans connaître leur signification.

4.2.2 Couche 2 — Transport & Messaging

Analogie directe avec la couche transport OSI (TCP/UDP). Elle gère le mode de transport des échanges (synchrone via REST/gRPC, ou asynchrone via Kafka), les stratégies de retry, les timeouts et l'idempotence des opérations. Le choix TCP $\equiv$ REST (fiable, ordonnée) ou UDP $\equiv$ Kafka (rapide, meilleur effort) est explicite dans la conception.

4.2.3 Couche 3 — Tenant & Routing

Analogie avec la couche réseau. Le *Tenant Resolver* joue le rôle d'un routeur : il détermine, à partir de l'en-tête de la requête (`tenant_id` ou sous-domaine), quelle instance de configuration utiliser, vers quel service router la requête et quelle version de l'API appliquer (versioning). C'est ici que réside la logique de découverte de service.

4.2.4 Couche 4 — Context & Policy

Analogie avec les couches session et présentation. Cette couche enrichit le contexte d'exécution : elle valide le token JWT, décode les rôles et permissions, applique les politiques de sécurité propres au tenant, détermine le niveau de SLA et initialise le contexte de saga pour les transactions longues. L'en-tête métier (H_C) est construit ici.

4.2.5 Couche 5 — Business Capabilities

Couche applicative. Elle contient les services métier génériques : gestion des acteurs, des ressources, des règles, des workflows, des documents et de l'audit. Ces services opèrent sur des modèles de données configurables par tenant, sans connaissance des couches inférieures.

4.3 Composants de l'architecture microservice

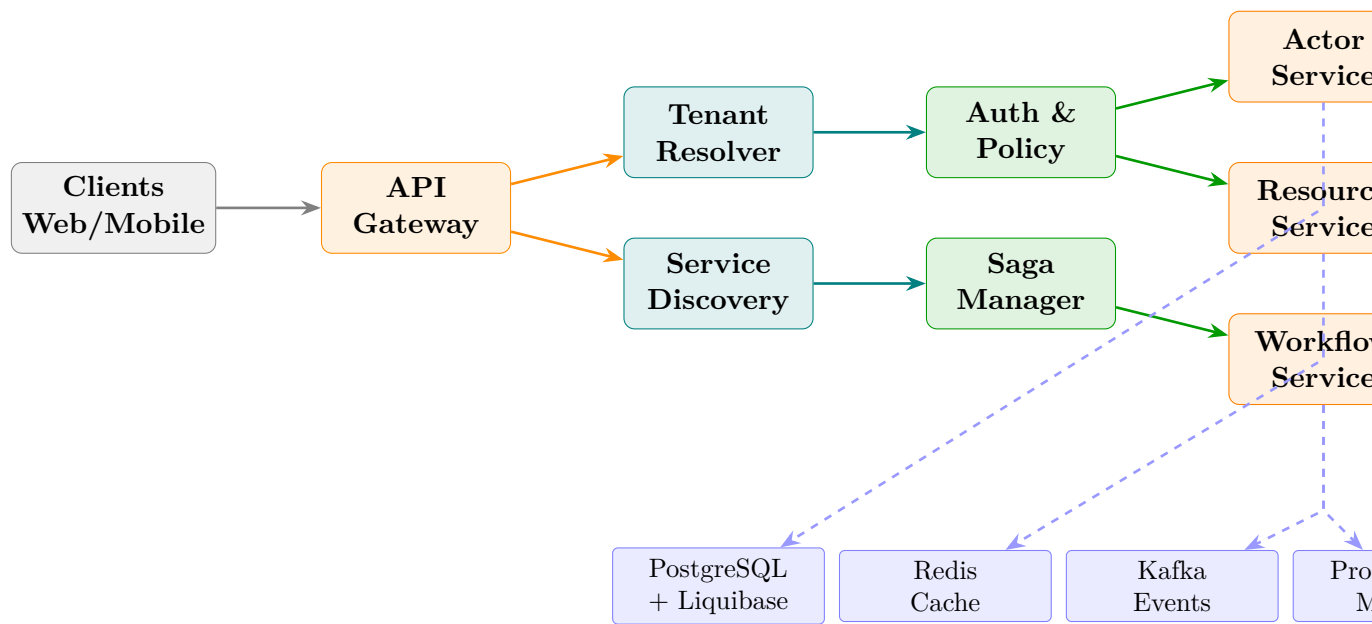


FIGURE 4.2 – Architecture microservice de référence de BCAAS

Chapitre 5

Conception détaillée

5.1 Modèle de données générique

Le défi central de BCAAS est de modéliser des entités métier hétérogènes (médecin, livre, appartement, produit agricole) avec un schéma de données unique. Nous adoptons le patron *Entity-Attribute-Value* (EAV) combiné à un catalogue de types.

5.1.1 Entités principales

TABLE 5.1 – Entités du modèle de données générique

Table	Rôle	Couche
tenants	Registre des instances métier	L3
tenant_configs	Configuration EAV par tenant	L3
actor_types	Catalogue des types d'acteurs	L5
actors	Instances d'acteurs par tenant	L5
resource_types	Catalogue des types de ressources	L5
resources	Instances de ressources par tenant	L5
business_rules	Règles métier paramétrables	L4
workflows	Définitions de processus	L5
workflow_instances	Exécutions en cours	L5
audit_logs	Journal des événements	L5
api_tokens	Tokens d'accès par tenant	L4

5.1.2 Diagramme entité-relation simplifié

5.2 Conception de l'API REST

L'API suit les principes REST avec versioning dans l'URL. Les ressources sont organisées selon la pile protocolaire.

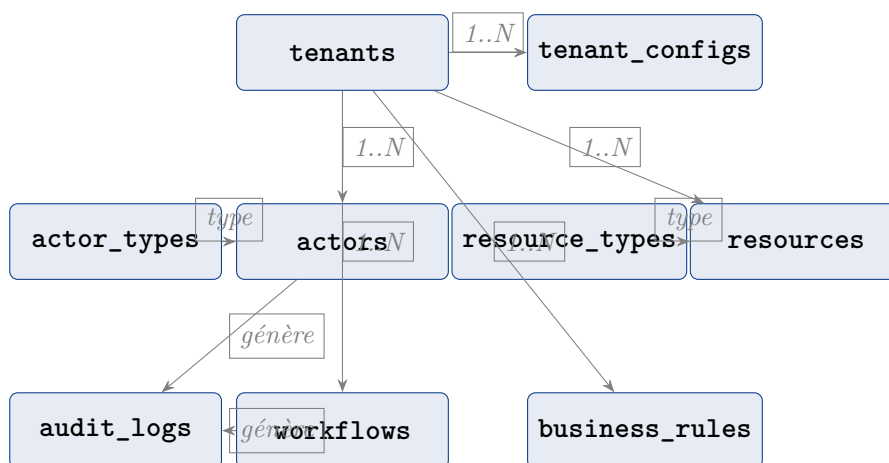


FIGURE 5.1 – Diagramme entité-relation du modèle générique

TABLE 5.2 – Endpoints principaux de l'API BCAAS

Méthode	Endpoint	Description	Couche
POST	/api/v1/tenants	Provisionnement d'un nouveau tenant	L3
GET	/api/v1/tenants/{id}	Lecture de la configuration tenant	L3
POST	/api/v1/auth/token	Émission d'un token JWT	L4
POST	/api/v1/actor-types	Création d'un type d'acteur	L5
GET	/api/v1/actor-types	Catalogue des acteurs	L5
POST	/api/v1/actors	Enregistrement d'un acteur	L5
POST	/api/v1/resource-types	Création d'un type de ressource	L5
POST	/api/v1/resources	Enregistrement d'une ressource	L5
POST	/api/v1/business-rules	Définition d'une règle métier	L4
POST	/api/v1/workflows	Définition d'un workflow	L5
GET	/api/v1/audit-logs	Consultation du journal	L5
GET	/api/v1/admin/stats	Statistiques globales	Adm.

5.3 Sécurité et authentification

La sécurité de BCAAS s'inspire directement du concept de firewall réseau avec filtrage à plusieurs niveaux.

1. **Niveau transport (L2)** : HTTPS obligatoire, TLS 1.3.
2. **Niveau routage (L3)** : Vérification du `tenant_id` dans chaque requête — analogie avec le filtrage par adresse IP source.
3. **Niveau politique (L4)** : Validation du token JWT(Spring Security + jjwt 0.12.6), vérification des rôles et permissions — analogie avec les listes de contrôle d'accès (ACL).
4. **Niveau applicatif (L5)** : Règles métier de validation (champs obligatoires,

contraintes de domaine).

La durée de vie du token JWT (TTL_{JWT}) est formellement analogue au champ TTL d'un paquet IP :

$$TTL_{JWT} = t_{emission} + \Delta t_{validite}, \quad \Delta t_{validite} \in [15\text{min}, 24\text{h}]$$

Chapitre 6

Implémentation technique

6.1 Pile technologique

TABLE 6.1 – Pile technologique de BCAAS

Composant	Technologie	Version
Backend Framework	Spring Boot	3.5.x
Langage	Java	21 LTS
Base de données	PostgreSQL	16
Migrations	Liquibase	4.x
Sécurité	Spring Security + JWT	jjwt 0.12.6
Documentation API	SpringDoc OpenAPI	2.8.6
Build	Maven	3.9+
Tests	JUnit 5 + Mockito	≥16 tests
Conteneurisation	Docker + Docker Compose	latest
Frontend	Next.js 14	PWA
Mobile	React Native	Expo

6.2 Structure du projet Spring Boot

Le projet suit l'architecture hexagonale (*ports et adaptateurs*) combinée avec le Domain-Driven Design (DDD). Cette organisation garantit que le domaine métier reste indépendant des détails techniques.

```
1      bcaas/  
2      +-- src/main/java/cm/bcaas/  
3      |   +-- domain/                # Couche 5 : entites et regles  
        metier  
4      |   |   +-- model/            # Entites du domaine  
5      |   |   +-- port/in/          # Interfaces d'entree (use  
        cases)
```

```

6      |      |      +-- port/out/                # Interfaces de sortie
      |      |      (repositories)
7      |      +-- application/                  # Couche 4 : orchestration,
      |      |      politiques
8      |      |      +-- service/               # Implementation des use cases
9      |      |      +-- saga/                  # Gestion des transactions
      |      |      longues
10     |      +-- infrastructure/
11     |      |      +-- persistence/           # Couche 1 : JPA, PostgreSQL
12     |      |      +-- web/                   # Couche 2-3 : controllers REST
13     |      |      |      +-- controller/
14     |      |      |      +-- filter/         # TenantFilter (L3)
15     |      |      |      +-- dto/
16     |      |      +-- security/              # Couche 4 : JWT, RBAC
17     |      +-- BcaasApplication.java
18     +-- src/main/resources/
19     |      +-- db/changelog/                 # Migrations Liquibase
20     |      +-- application.yml
21     +-- src/test/                           # JUnit 5 + Mockito

```

Listing 6.1 – Structure du projet BCaaS

6.3 Filtre de résolution du tenant

Le composant central de la couche L3 est le `TenantFilter`, qui joue le rôle de routeur réseau : il lit l'identifiant de tenant dans l'en-tête de la requête et configure le contexte d'exécution.

```

1      @Component
2      @Order(1)
3      public class TenantFilter extends OncePerRequestFilter {
4
5          @Override
6          protected void doFilterInternal(HttpServletRequest req,
7              HttpServletResponse res, FilterChain chain)
8              throws ServletException, IOException {
9
10             // Lecture du tenant_id (analogie : lecture de l'adresse IP
11                destination)
12             String tenantId = req.getHeader("X-Tenant-ID");
13
14             if (tenantId == null || tenantId.isBlank()) {
15                 res.sendError(HttpServletResponse.SC_BAD_REQUEST,
16                     "En-tete X-Tenant-ID manquant");
17                 return;
18             }
19         }
20     }

```

```

18      // Resolution du tenant (analogie : resolution DNS / table de
19      routage)
20      TenantContext.setCurrentTenant(tenantId);
21
22      try {
23          chain.doFilter(req, res);
24      } finally {
25          // Nettoyage du contexte (analogie : fermeture de session)
26          TenantContext.clear();
27      }
28  }
29  }

```

Listing 6.2 – TenantFilter — Couche 3 (Tenant & Routing)

6.4 Structure du paquet métier (DTO)

```

1  /**
2   * Representation du "paquet metier" -- analogie avec le datagramme
3   * H_T : en-tete de transport / H_C : en-tete de contexte /
4   * payload : donnees
5   */
6  public record BusinessPacket<T>(
7      // En-tete de transport (analogie : en-tete IP)
8      TransportHeader transportHeader,
9      // En-tete de contexte metier (analogie : en-tete applicatif)
10     ContextHeader contextHeader,
11     // Payload metier (analogie : SDU -- Service Data Unit)
12     T payload
13 ) {
14     public record TransportHeader(
15         String version,    // Version de l'API
16         String method,    // Methode HTTP
17         Instant timestamp // Horodatage
18     ) {}
19
20     public record ContextHeader(
21         String tenantId,    // Identifiant du tenant
22         String actorId,    // Identifiant de l'acteur
23         String traceId,    // Identifiant de correlation (analogie
24         : trace reseau)
25         String roleScope,  // Portee des permissions
26         SlaLevel slaLevel // Niveau de QoS contractualise
27     ) {}

```

Listing 6.3 – BusinessPacket — Structure du paquet métier

6.5 Pertinence de Kafka et Redis dans BCaaS

Kafka et Redis : pertinents pour BCaaS ?

Kafka (analogue UDP/protocole de diffusion) est pertinent pour les notifications asynchrones, l'audit en temps réel et la propagation d'événements entre instances métier. Il n'est pas indispensable dans un premier prototype mais devient essentiel dès que plusieurs tenants génèrent du volume.

Redis (analogue au cache ARP) accélère la résolution des configurations tenant (évite une requête PostgreSQL à chaque appel) et stocke les tokens invalidés (*blacklist*). Son ajout améliore significativement les performances sans changer l'architecture.

Recommandation : intégrer Redis en priorité (impact fort, coût faible), Kafka en deuxième phase.

Chapitre 7

Frontend Next.js

7.1 Architecture du frontend

Le frontend de BCAAS est divisé en deux parties distinctes correspondant aux deux plans de l'architecture :

Site vitrine (data plane view) Présentation publique de l'API, documentation interactive, exemples d'utilisation et catalogue des instances métier disponibles.

Tableau de bord Admin (control plane) Interface sécurisée pour la gestion des tenants, le suivi des appels API, les statistiques d'utilisation et la configuration des règles.

7.2 Choix techniques

TABLE 7.1 – Technologies frontend

Besoin	Technologie	Justification
Framework	Next.js 14 (App Router)	SSR/SSG, SEO, PWA native
Styling	Tailwind CSS	Rapidité, cohérence design
UI Components	shadcn/ui	Accessibilité, personnalisable
Graphiques admin	Recharts	Léger, intégrable React
Auth client	NextAuth.js	OAuth2 / JWT intégré
État global	Zustand	Simple, performant
Internationalisation	next-i18next	Support multi-langue

7.3 Pages principales

7.3.1 Site vitrine

- / — Hero section avec présentation de l'approche protocolaire
- /architecture — Diagramme interactif des couches
- /docs — Documentation API (intégration Swagger UI)
- /instances — Galerie des instances métier disponibles
- /getting-started — Guide de provisionnement tenant

7.3.2 Tableau de bord admin

- /admin/dashboard — Vue globale : tenants actifs, appels/min, erreurs
- /admin/tenants — Gestion CRUD des tenants
- /admin/tenants/[id] — Détail : configuration, acteurs, ressources, règles
- /admin/api-calls — Journal des appels API avec filtres
- /admin/stats — Graphiques : volume, latence, répartition par tenant

Chapitre 8

Tests et validation

8.1 Stratégie de tests

La stratégie de tests suit également la logique en couches :

Tests unitaires (couches L4-L5) Validation des règles métier, des politiques et des services avec JUnit 5 + Mockito. Objectif : ≥ 16 tests passants.

Tests d'intégration (couches L2-L3) Validation des filtres, du routage tenant et de la chaîne d'authentification avec Spring Boot Test.

Tests de contrat API Validation des schémas OpenAPI avec Spring MVC Test + Swagger Validator.

Tests de performance Simulation de charge multi-tenant avec JMeter ou k6.

8.2 Scénario de test de provisionnement tenant

```
1      @SpringBootTest(webEnvironment = RANDOM_PORT)
2      @AutoConfigureMockMvc
3      class TenantProvisioningIntegrationTest {
4
5          @Test
6          void shouldProvisionNewTenantAndReturnConfig() throws Exception
7          {
8              // Given
9              var request = ""
10             {
11                 "name": "Pharmacie Centrale",
12                 "domain": "pharmacy",
13                 "slaLevel": "STANDARD",
14                 "locale": "fr-CM"
15             }
16         }
```

```
15         """;
16
17         // When - analogie : envoi d'un paquet reseau
18         mockMvc.perform(post("/api/v1/tenants")
19             .contentType(APPLICATION_JSON)
20             .content(request))
21         // Then - analogie : accuse de reception (ACK)
22             .andExpect(status().isCreated())
23             .andExpect(jsonPath("$.tenantId").exists())
24             .andExpect(jsonPath("$.apiKey").exists())
25             .andExpect(jsonPath("$.status").value("ACTIVE"));
26     }
27 }
```

Listing 8.1 – Test d'intégration — Provisionnement tenant

Chapitre 9

Conclusion et perspectives

9.1 Bilan

Ce projet a démontré qu’une analogie rigoureuse entre l’ingénierie des protocoles réseaux et la conception d’une plateforme métier n’est pas une simple métaphore pédagogique : elle constitue une méthode de conception puissante. Les concepts de couches, d’encapsulation, de routage, d’adressage, de QoS et de séparation control/data plane ont guidé chaque décision architecturale de BCAAS.

Les réalisations principales sont :

- Une pile protocolaire métier en cinq couches formellement liée au modèle OSI.
- Un modèle de données générique et configurable par tenant.
- Une API REST documentée et versionnée avec Spring Boot 3.5 / Java 21.
- Une authentification JWT multi-tenant avec contrôle d’accès basé sur les rôles.
- Un frontend Next.js avec site vitrine et tableau de bord d’administration.

9.2 Perspectives

Court terme Intégration de Redis pour le cache des configurations tenant et Kafka pour la propagation d’événements asynchrones.

Moyen terme Migration de l’architecture modulaire monolithique vers des micro-services déployés individuellement (cible définie dans le cahier des charges).

Long terme Marketplace d’instances métier permettant à des entreprises camerounaises et africaines de déployer leur propre configuration BCAAS sans développement.

9.3 Apports personnels

Ce projet a permis d’acquérir des compétences concrètes en architecture logicielle distribuée, en conception d’API REST professionnelle, en sécurité applicative (JWT, RBAC) et en modélisation de systèmes génériques. La démarche de pensée protocolaire — définir les contrats avant le code, modéliser les échanges comme des messages — constitue un réflexe d’ingénieur directement applicable à tout projet de systèmes d’information.

Bibliographie

- [1] A. S. Tanenbaum, D. J. Wetherall, *Computer Networks*, 5th ed., Pearson, 2011.
- [2] M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.
- [3] S. Newman, *Building Microservices*, 2nd ed., O'Reilly, 2021.
- [4] E. Evans, *Domain-Driven Design : Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003.
- [5] C. Richardson, *Microservices Patterns*, Manning, 2018.
- [6] SpringDoc OpenAPI Documentation, <https://springdoc.org>, consulté en 2025.
- [7] M. Jones, J. Bradley, N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, 2015.
- [8] ISO/IEC 7498-1, *Information technology — Open Systems Interconnection — Basic Reference Model*, ISO, 1994.